

Blockchain-based Firmware Update Scheme Tailored for Autonomous Vehicles

Mohamed Baza*, Mahmoud Nabil*, Nouredine Lasla[§], Kemal Fidan[‡],
Mohamed Mahmoud*, Mohamed Abdallah[§]

*Department of Electrical and Computer Engineering, Tennessee Tech University, Cookeville, TN, USA

[§]Division of Information and Computing Technology, College of Science and Engineering, HBKU, Doha, Qatar

[‡]Department of Electrical Engineering and Computer Science, University of Tennessee, Knoxville, TN, USA

Abstract—Recently, Autonomous Vehicles (AVs) have gained extensive attention from both academia and industry. AVs are a complex system composed of many subsystems, making them a typical target for attackers. Therefore, the firmware of the different subsystems needs to be updated to the latest version by the manufacturer to fix bugs and introduce new features, e.g., using security patches. In this paper, we propose a distributed firmware update scheme for the AVs' subsystems, leveraging blockchain and smart contract technology. A consortium blockchain made of different AVs manufacturers is used to ensure the authenticity and integrity of firmware updates. Instead of depending on centralized third parties to distribute the new updates, we enable AVs, namely distributors, to participate in the distribution process and we take advantage of their mobility to guarantee high availability and fast delivery of the updates. To incentivize AVs to distribute the updates, a reward system is established that maintains a credit reputation for each distributor account in the blockchain. A zero-knowledge proof protocol is used to exchange the update in return for a proof of distribution in a trustless environment. Moreover, we use attribute-based encryption (ABE) scheme to ensure that only authorized AVs will be able to download and use a new update. Our analysis indicates that the additional cryptography primitives and exchanged transactions do not affect the operation of the AVs network. Also, our security analysis demonstrates that our scheme is efficient and secure against different attacks.

Index Terms—Firmware update, Blockchain, Smart contracts, Autonomous Vehicles, Attribute based encryption (ABE), Zero-knowledge proof.

I. INTRODUCTION

A. Motivation

Over the last few years, the automobile industry has achieved a notable leap towards the realization of practical Autonomous Vehicles (AVs). AVs are equipped with sophisticated systems and subsystems to provide vehicles with advanced communication capabilities, computer vision, autonomous decision-making capability, etc., to enable them to autonomously drive without any human intervention [1]. AVs have the potential to enhance our current transportation system by reducing congestion and travel time, increasing fuel efficiency, and improving road safety [2].

AVs are composed of many subsystems running specific firmware programs that enable performing all control, monitoring, and data manipulation operations. However, by controlling the functionality of the subsystems through the installation of infected versions of the corresponding firmwares, an

attacker can successfully hack AVs and fully/partially access them, e.g., to involve the vehicle in accidents deliberately, which may lead to dramatic damages and kill people. As an example of this attack, Chrysler company announced a recall for 1.4 million vehicles after hackers have managed to turn-off the engine remotely while the vehicles were on motion by exploiting a hackable software vulnerability via the internet-connected entertainment system [3]. Therefore, ensuring the integrity and authenticity of AVs' firmware update is primordial and must be carefully addressed. In addition, it may happen that multiple AVs with their various subsystems need to be updates urgently and simultaneously, e.g., to fix newly discovered bugs, thus a high availability of the updates is required.

Most of the existing solutions for firmware update depends on the client-server model in which a manufacturer delegates the process of firmware distribution to trusted cloud providers, such as Microsoft Azure and IBM Cloud [4]. However, this central client-server architecture has the single point of failure problem. In case the server is not available, the clients (AVs) cannot access the resources (updates) no matter how powerful the server is. For AVs, there are several factors that make the availability and security of the firmware updates challenging tasks. To elaborate, the number of autonomous vehicles on roads is expected to reach 20.8 million in the U.S. alone [4]. Also, each AV has many subsystems that run different programs designed to accomplish specific functions. This creates tremendous load on the server side and can broaden the sources of cyberattacks. Moreover, AVs are designed to last for many years (15 to 20 years); thus the integrity and authenticity of the firmware should be guaranteed throughout the years of service.

Recently, blockchain with its capability to provide a verified, transparent and distributed ledger without a need of a trusted third party, has drawn the attention of both academia and industry across a wide range of domains, including healthcare, finance, and energy [5]. Moreover, blockchain paved the way to build smart contracts, which serve as a piece of code on the blockchain that can perform an action once specific criteria are satisfied. More importantly, self-enforcing smart contracts can be executed without the need for trusted intermediaries [5], [6].

B. Contribution

In this paper, we propose a blockchain-based firmware update scheme tailored for AVs. We use blockchain and smart-contract technology to guarantee the authenticity and integrity of new updates. We also exploit the AVs' inter-communication capability and incentivize AVs to participate in the distribution and transfer of new firmware updates from one to another therefore ensuring, high availability and fast delivery of the updates. The main contributions of the proposed scheme are outlined as follows:

- A consortium blockchain created by multiple manufacturer organizations is proposed. Each consortium member has the permission to write a smart contract that handle the logic ensuring the authenticity and integrity of its firmware updates without the need for a trusted third party.
- A high availability and reliability is ensured by incentivizing AVs to participate in the distribution of the firmware updates. Distributor AVs are rewarded for their honest participation and the smart contract is used to manages the reward system and keeps track of the reputation credit of each AV.
- Attribute-based encryption (ABE) technique is used to allow manufacturers to set a policy about who have the rights to download and use an update. The access policy is defined on the smart contract that enforces its execution without an intermediary, so only authorized AVs can request and receive the update.
- Since AVs do not mutually trust each other, a Zero-Knowledge Proof protocol is employed. Each distributor can exchange an encrypted version of the update in return for proofs of distribution from receiver AVs. The delivery of the decryption key is guaranteed by the smart contract which will reveal the key once the proofs are collected. The smart contract also increment the distributor's reputation based on the received proofs.

The rest of this paper is organized as follows. In Section II, we discuss some preliminaries. Then, our proposed scheme is presented in details in Section III. Performance evaluations are provided in Section IV. Section V discusses the related work. Finally, we give concluding remarks in Section VI followed by acknowledgement in Section VII.

II. PRELIMINARIES

In this section, we present the necessary background on blockchain, smart contracts and some cryptographic tools that we have used for this research.

A. Blockchain and Smart Contracts

Blockchain was first introduced in 2008 as the underline technology behind the cryptocurrency known as Bitcoin [7] to help make peer-to-peer exchange of value without a centralized third party. A blockchain is a distributed, immutable, and append-only data structure formed by a sequence of blocks that are chronologically and cryptographically linked together [8]. Fundamentally, a network composed of a set of

nodes called miners or validators are responsible of keeping a trustworthy record of all transactions through a consensus algorithm in a trust-less environment. To exchange some coins from one account to another, for instance, a new transaction is generated and broadcast to the network. Each user is identified by a pseudonym address, usually generated from its public key, and the transaction is authenticated through a digital signatures computed using the user's private key. One of the exciting applications of blockchains is smart contracts, which are defined as computer codes running on top of a blockchain and is correctly executed without fraud or any interference from a third party. Each contract has a unique address on the blockchain to identify itself and to allow users or other contracts to interact with it. The most popular smart contracts platform is Ethereum, and the de-facto language for creating contracts in Ethereum is Solidity [8].

B. Cryptographic Tools

1) *Attribute Based Encryption*: Attribute based encryption (ABE) is an encryption scheme that allows access control over encrypted data. In ABE, each user is assigned a set of secret keys corresponding to his/her set of attributes. Then, a message is encrypted under an access policy formed from the system's set of attributes. The message can only be decrypted by the users who have the attributes that can satisfy the policy. In our scheme, we use the attribute based encryption scheme proposed in [9] to enable the distributor AV to identify the neighbouring AVs who have the required features to download a firmware update. This scheme is a ciphertext-policy attribute-based encryption (CP-ABE), where the access policy is embedded in the ciphertext.

2) *Zero-Knowledge Succinct Non-Interactive Argument of Knowledge (zk-SNARK)*:: zk-SNARK is a proof construction in which one, called the prover, can prove possession of a specific information, called a witness (w), e.g., a secret key, to someone else, called the verifier, without revealing that information. zk-SNARK does not require any interaction between the prover and verifier. Moreover, these schemes are efficient in the sense that the zero-knowledge proof can be verified quickly.

We adopt the zk-SNARK scheme in [10]. Formally speaking, let L be an NP language with C as its decision circuit. Two keys play an essential role, namely, the proving key ($\mathcal{PK}$) and the verifying key ($\mathcal{VK}$). The proving key allows any prover to compute a proof π for a statement $y \in L$ with a witness w . Typically, a zk-SNARK scheme consists of the following three polynomial-time algorithms:

- 1) $Gen(1^\lambda, C) \rightarrow (\mathcal{PK}, \mathcal{VK})$. Given a security parameter λ and C as a decision circuit, the *Gen* algorithm generates two public keys, including $\mathcal{PK}$ and $\mathcal{VK}$, that are used to prove/verify the membership in L .
- 2) $Prove(\mathcal{PK}, y, w) \rightarrow \pi$: Given $\mathcal{PK}$, instance y , and witness for a NP statement w , the *Prove* algorithm generates a proof π for the statement $x \in L_C$.

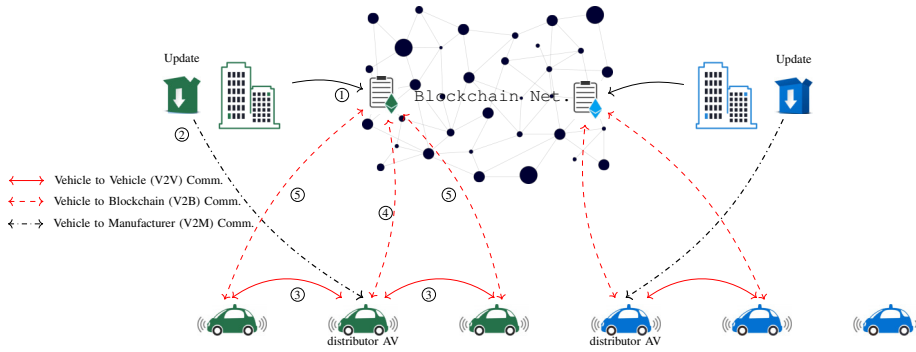


Fig. 1: System architecture: (1) The manufacturer creates a smart-contract for a new firmware update by including its hash code for authenticity checking by AVs. (2) The manufacturer sends the new update to top-reputation AVs (distributors). (3) A distributor exchanges an encrypted version of the update in return for proof of reception of the update by a responder AV. (4) A redeem transaction, containing multiple proofs, is sent to the smart contract to update the distributor's reputation. (5) The responder AV receives the decryption key of the firmware update from the smart contract.

- 3) $Verify(\mathcal{VK}, y, \pi) \rightarrow \{0, 1\}$. Given $\mathcal{VK}$, instance y , and the proof π , the *Verify* algorithm outputs 1 if $y \in L_c$, allowing the verifier to verify the instance y .

III. SECURE AND SCALABLE FIRMWARE UPDATE SCHEME

In this section, we present our scheme that aims to ensure secure and scalable delivery of firmware updates from automobile manufacturers to AVs. We first present a general architecture for the system, followed by system initialization, the smart contract creation, firmware update dissemination, and rewarding.

A. System Architecture

Fig. 1 shows the system architecture, which is comprised of two manufacturers with their AVs, two smart contracts for firmware updates of each manufacturer, and a consortium blockchain. A sketch of the possible interactions between the different system entities is shown in Fig. 2. The role of each entity in the system is discussed in the following paragraphs.

Manufacturer. The manufacturer is responsible for keeping its manufactured AVs updated with the latest versions of the different firmware updates for the subsystems that control the AVs. During the manufacturing of AVs, the manufacturer uploads each AV with a set of cryptographic keys and public parameters that will be used to ensure secure distribution of firmware updates. Also, each time a new update is released, a corresponding smart contract is deployed by the manufacturer to allow AVs to check the integrity and authenticity of the update. In addition, to attract AVs to participate in the distribution of an update, the manufacturer compensates the participants through a rewarding mechanism, e.g., momentary rewards and free or reduced-price maintenance services.

Autonomous Vehicles. We distinguish between two types of AVs, *distributors* and *responders*. The distributor AV disseminates a new firmware update to other AVs (responders) in its vicinity. Each responder AV that receives an update can also act as a distributor of that update. By this way, we can

ensure the large-scale dissemination of the update quickly. Initial distributors are selected by the manufacturer based on their reputations which are recorded in a smart-contract.

Smart Contract. For each new firmware update, a smart contract is created. The contract contains the necessary credentials allowing any receiver of the update to authenticate it and verify its integrity. In addition, the contract implements the reputation logic that evaluates and keeps track of the AVs' activities in the distribution of the firmware update. More specifically, the contract increases the reputation of a distributor AV after receiving proofs of the firmware distributions from responder AVs.

Blockchain Network. Blockchain network is at the center of our system and it executes the smart contracts in a distributed manner without relying on a central party. This is mandatory to ensure a scalable and secure firmware update dissemination. Moreover, to improve the efficiency of the system, we opt for a *consortium blockchain*, where the validators, i.e., nodes with write permission on the shared ledger, are known and trusted entities. In our case, the validators can be the manufacturers of the different automobile brands.

B. System Initialization

A multi-authority attribute based encryption scheme is used, where each manufacturer is considered as an authority that decides a set of attributes (or features) for its AVs. $\mathbb{M}$ is the set of all available manufacturers and a manufacturer $\mathcal{M}_\theta \in \mathbb{M}$. Let $\mathbb{A}$ be the set of all attributes (or features) in the system, an access policy (A, δ) on $\mathbb{A}$ with $A \in \mathbb{Z}_p^{l \times n}$, called the share generating matrix in the field $\mathbb{Z}_p$ of prime order p with l rows and n columns, and a function δ that labels the rows of A with attributes from $\mathbb{A}$, i.e., $\delta: [l] \rightarrow \mathbb{A}$. In addition, let ρ be a function that maps attributes in rows to its manufacturers, where $\rho: [l] \rightarrow \mathcal{M}_\theta$. Consider $e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ a cryptographic bilinear map with generators $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$, where $\mathbb{G}_1$ and $\mathbb{G}_2$ are multiplicative group. Each manufacturer

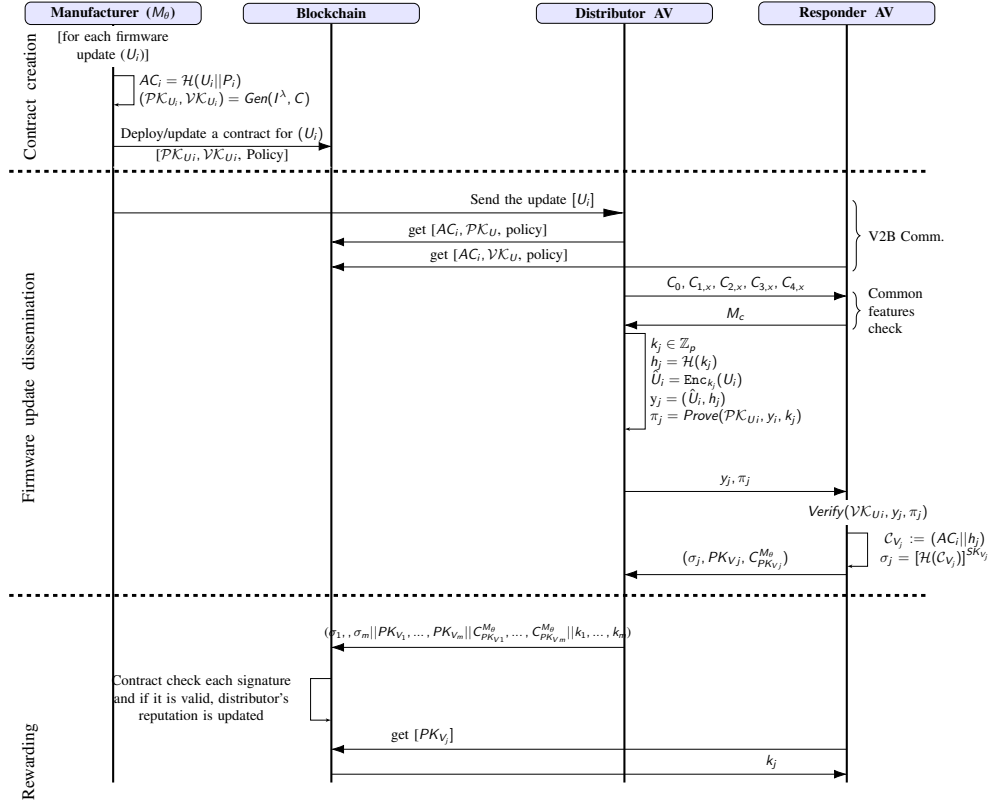


Fig. 2: Firmware update scheme UML diagram.

$\mathcal{M}_\theta \in \mathbb{M}$ should select two random elements $(\alpha_\theta, \gamma_\theta) \xleftarrow{R} \mathbb{Z}_p^*$ as its secret keys, and then, $\mathcal{M}_\theta$ can compute its public key as $PK_\theta = \{e(g_1, g_2)^{\alpha_\theta}, g_1^{\gamma_\theta}\}$. Besides, a public hash function $\mathcal{H} : \{0, 1\}^* \rightarrow G_1$ is used to map an AV global identifier GID to a point in G_1 , public hash function $F : \{0, 1\}^* \rightarrow G_1$ that maps an attribute $a \in \mathbb{A}$ to G_1 , and a function T that maps an attribute $a \in \mathbb{A}$ to the manufacturer $\mathcal{M}_\theta$, hence, the function $\rho(\cdot)$ can be redefined as $\rho(\cdot) : T(\delta(\cdot))$. The global parameters are then defined as $GP = \{G_1, G_2, G_T, \mathbb{Z}_p, \mathcal{H}, F, T, \mathbb{A}, \mathbb{M}\}$.

Besides, during the production, $\mathcal{M}_\theta$ should assign each AV a key for each assigned attribute $a \in \mathbb{A}$ using the AV global identity GID as follows: $\mathcal{M}_\theta$ chooses a random $t \xleftarrow{R} \mathbb{Z}_p^*$ and outputs to the AV attributes secret keys as $SK_{GID,a} = \{K_{GID,a} = g_2^{\alpha_\theta} \mathcal{H}(GID)^{\gamma_\theta} F(a)^t, K'_{GID,a} = g_1^t\}$. Finally, for each AV, $\mathcal{M}_\theta$ generates a public/private key pair as follows: a random number $x_a \xleftarrow{R} \mathbb{Z}_p$ is selected as the private key and the corresponding public key is $PK_{V_j} = g_2^{x_a}$. The public key PK_{V_j} should be associated with a manufacturer's certificate as $(C_{PK_{V_j}}^{M_\theta})$. Then, the public/private key pair (PK_{V_j}, x_a) and $C_{PK_{V_j}}^{M_\theta}$ should be added to the AV's tamper proof device along with manufacturer's public key PK_θ .

C. Smart Contract Creation

Upon releasing a new update by a subsystem's manufacturer, denoted by U_i , the AV manufacturer that uses the subsystem should first test the update. Note that a subsystem's manufacturer may be different from the AV manufacturer. If the AV manufacturer decides to use it on its AVs, it starts the firmware update as follows. The manufacturer creates a smart contract and initializes it by two attributes: (1) A proving/verifying key pair: $(PK_{U_i}, VK_{U_i}) = \text{Gen}(1^\lambda, C)$, required for the execution of the zk-SNARK protocol; (2) An authentication code for the new firmware update: $AC_i = \mathcal{H}(U_i || P_i)$, where P_i is the access policy defined by the manufacturer to deliver the update to only AVs that have the features defined in the policy.

The manufacturer deploys a smart-contract by broadcasting a transaction to the blockchain network. The deployed smart-contract is described in Algorithm 1 and includes the following main functions:

- *Authenticity and integrity of a firmware update.* Since the update's authentication code and verification key are stored in the contract, AVs, by consulting the blockchain, can check whether a received firmware update is the same one that was originally approved by the AV manufacturer.
- *AVs' reputation.* When an AV participates in the distribu-

Algorithm 1: Pseudocode for *Firmware Update* contract

```

1 contract FirmwareUpdate
2   mapping(address => int) Reputation
   // Mapping for distributors
3   mapping(address => int) UpdatedAVs
   // Mapping for Updated AVs
4   function FirmwareUpdate(_PK, _VK, _ACi,
   _Pi, X)
5     PK ← _PK // Proving Key
6     VK ← _VK // Verifying key
7     ACi ← _ACi // authentication code
8     Pi ← _Pi // ABE Policy
9     MaxUpdate ← X // Max. No. of download
10  function RecieveProof(σ[], PK[], C[]
  keys[])
11    address [] RecievedAVs // Received AV
12    for s ← 0 to PK.length do
13      if verifySig(pk_M, PK[s], C[s]) or
        UpdatedAVs[PK[s]] > MaxUpdate
14        return
15      end
16      hs ← H(keys[s])
17      CVs ← H(ACi, hs)
18      if verifySig(CVs, σs, PK[s])
19        Reputation[Dist] ←
          Reputation[Dist] + 1
          // increase Av reputation
20        emitEvent("KeyRevealed", PKi,
          keys[i])
21        UpdatedAVs[PKi] ←
          UpdatedAVs[PKi] + 1
22      end
23    end

```

tion of a new update, the proof of distribution is sent to the smart contract which in turn increases its reputation. The manufacturer rewards the highly-reputed AVs, i.e., the active AVs in distributing the firmware. The reward can be momentary, free or reduced-price maintenance service, etc.

- *Firmware access control.* Each firmware update has an access policy set by the manufacturer, and the AVs that have enough features to satisfy the policy can receive the firmware. This can restrict the distribution of the firmware to only certain AVs. The access policy of an update is included in the update's smart-contract.

D. Firmware Update Dissemination

In this stage, the manufacturer starts the dissemination process of a new update by first selecting the most active AVs in distributing updates (based on their reputations) to act as the initial distributors. As discussed before, rewards are used to incentivize the AVs to act as distributors and actively distribute the new firmware, but the rewarding system should be secure to ensure that only honest distributors which distributes the firmware are rewarded. In practice, each AV manufacturer will use many subsystems made by different companies, and therefore, it is very frequent that different AV manufacturer may use the same subsystems produced by the

same company in their vehicles. Thus, it is very important for each manufacturer to ensure that its distributors will deliver a particular update to only certain models of its AVs.

Thanks to ABE, as presented in Section II, each manufacturer can define an access policy for each update on the associated smart contract, where only AVs that belong to the same manufacturer and have enough fractures, such as model, year of manufacturer, etc, can decrypt and use the firmware update they got from a distributor. For a distributor to find other AVs which can satisfy the access policy of an update and deliver it, the following steps should be taken.

- 1) A distributor AV first queries the blockchain for the AC_i, proving key PK_{U_i}, and the manufacture's access policy (A, δ).
- 2) Then, distributor AV should broadcast an encrypted challenge message (M_c) using the ABE to the nearby AVs. This message is encrypted using the manufacture's public key PK_θ under the access policy (A, δ) set by the manufacturer. Hence, only the AVs which owns the set of attributes that satisfy the policy are able to decrypt the ciphertext CT. To encrypt M_c, distributor AV first creates two random vectors $v = (z, v_2, \dots, v_n)^T$ and $w = (0, w_2, \dots, w_n)^T$, where $\{z, v_2, \dots, v_n, w_2, \dots, w_n\}$ are elements randomly selected from $\mathbb{Z}_p^*$. We denote λ_x as the share of the random secret z corresponding to row x, i.e., $\lambda_x = (A_x \cdot v)$ and w_x denotes the share of zero, i.e., $w_x = (A_x \cdot w)$, where A_x is the x-th row of access matrix A. The distributor AV chooses a random element $t_x \xleftarrow{R} \mathbb{Z}_p^*$ for each row in the policy matrix A and computes the CT as:

$$\begin{aligned}
 C_0 &= M_c \cdot e(g, g)^z; \\
 \{C_{1,x}\} &= e(g_1, g_2)^{\lambda_x} e(g_1, g_2)^{\alpha_{\rho(x)} t_x}; \\
 C_{2,x} &= g_1^{-t_x}; \\
 C_{3,x} &= g_1^{y_{\rho(x)} t_x + w_x}; \\
 C_{4,x} &= F(\delta(x))^{t_x} \}_{x \in [I]}
 \end{aligned}$$

- 3) After a responder AV receives CT, it first queries the smart contract for the access policy (A, δ), VK_{U_i} and AC_i. Then, to decrypt M_c, the AV should use the policy (A, δ) from the blockchain and its secret keys (K_{GID,a}, K'_{GID,a}) for the subset of rows A_x of satisfied attributes and for each row x to compute

$$C_{1,x} \cdot e(K_{GID,\delta(x)}, C_{2,x}) \cdot$$

$$e(H(GID), C_{3,x}) \cdot e(K'_{GID,\delta(x)}, C_{4,x})$$

$$= e(g_1, g_2)^{\lambda_x} \cdot e(H(GID), g_2)^{w_x}$$

Then, AV calculates the constants $c_x \in \mathbb{Z}_p^*$ such that $\sum_x c_x A_x = (1, 0, \dots, 0)$ and computes:

$$\prod_x (e(g, g)^{\lambda_x} \cdot e(H(GID), g_2)^{w_x})^{c_x} = e(g, g)^z$$

This is true because $\lambda_x = (A_x \cdot v)$ and $w_x = (A_x \cdot w)$, where $\langle (1, 0, \dots, 0) \cdot v \rangle = z$ and $\langle (1, 0, \dots, 0) \cdot w \rangle = 0$. Hence, the challenge message can be decrypted as $M_c = C_0 / e(g, g)^z$.

- 4) Finally, once a responder AV manages to get M_c, it then replies to the distributor AV with the correct M_c. Henceforth, both the

distributor and responder AV can proceed with the firmware update transfer.

A distributor AV sends the firmware update to a responder AV in return for a signature, i.e., a proof for disseminating the firmware. This exchange of firmware update and proof can be made in a trust-less way using zk-SNARK protocol as follows.

- 1) The distributor AV generates a secret key $k_j \in \mathbb{Z}_P$ and calculates $h_j = \mathcal{H}(k_j)$.
- 2) Then, it computes $\hat{U}_i = \text{Enc}_{k_j}(U_i)$, where Enc is a symmetric-key encryption algorithm.
- 3) For zk-SNARK protocol, the secret witness is the instance $y_j = (\hat{U}_i, h_j)$ and k_j . The NP statement is as follows:

$$\exists k_j : \mathcal{H}(k_j) = h_j \wedge \mathcal{H}(\text{Dec}_{k_j}(\hat{U}_i), P_i) = AC_i \quad (1)$$

Which attests that the distributor AV has a key, k_j , such that its hash is h_j , and if k_j is used to decrypt $\hat{U}_i$, it will match the update authentication code AC_i . After that, the distributor computes a zero-knowledge proof $\pi_j = \text{Prove}(\mathcal{PK}_{U_i}, y_j, h_j)$ and sends $y_j || \pi_j$ to the responder.

- 4) Upon receiving $(y_j || \pi_j)$, the responder first verifies that $\text{Verify}(\mathcal{VK}_{U_i}, y_j, \pi_j) = 1$, and then computes a signature $\sigma_j = [\mathcal{H}(C_{V_j})]^{SK_{V_j}}$, where SK_{V_j} is its private key and $C_{V_k} = (AC_i, h_j)$.
- 5) Finally, it sends $(\sigma_j || PK_{V_j} || C_{PK_{V_j}}^{M_\theta})$ to the distributor.

E. Rewarding

In this phase, the distributor AV sends a redeem transaction, containing multiple proofs, to the smart contract to update its reputation proportionally to the number of AVs which received the update. This rewarding process is done as follows.

- 1) Once the distributor AV gets the proof of transferring an update (σ_j) from other vehicle,
- 2) The distributor sends a transaction to the blockchain containing $(\sigma_{V_1}, \dots, \sigma_{V_m} || PK_{V_1}, \dots, PK_{V_m} || C_{PK_{V_1}}^{M_\theta}, \dots, C_{PK_{V_m}}^{M_\theta} || k_1, \dots, k_m)$ where m is the number of vehicles that received the update.
- 3) The smart contract method `RecieveProof` first verifies that the received public key is one of the certified keys by the manufacturer (see line 10 in Algorithm 1) as well as many number of times that AV gets the update. Then, it computes $h_j = \mathcal{H}(k_j)$ and $C_{V_j} = \mathcal{H}(AC_i, h_j)$ for all j . Thereafter, it verifies the signature of each AV that have received the update (see line 13 in Algorithm 1).
- 4) Finally, the distributor is rewarded by increasing its reputation index proportionally to the number of vehicles that received the update (see the method `UpdateReputation` in Algorithm 1).
- 5) A responder AV queries the contract for a relevant event associated with its public key for the decryption key k_j it needs to decrypt $\hat{U}_i$ to get U_i (see event "KeyRevealed" in Algorithm 1).

IV. PERFORMANCE ANALYSIS

A. Performance Evaluation

In this section, we evaluate the computation overhead for the cryptography operations used in our scheme.

We measured the computation times of ABE using Intel Core i7- 4765T 2.00 GHz and 8GB RAM machine and Python charm cryptographic library [11]. In our scheme, a distributor AV needs to broadcast a challenge packet (M_c) encrypted by a number of attributes (γ) specified in the smart contract. the required time for encryption is $(10.9 \times \gamma + 1.35)$ ms. In addition, a responder AV needs to decrypt (M_c) with total decryption time that can be formulated as $(4.03 \times \gamma + 0.01)$ ms. After running the ABE scheme, zk-SNARK protocol should be run. In this protocol, a proof is generated by the distributor and then verified by the responder. We implemented the NP statement in Equation 1 using Zokrates toolbox [12]. The time to generate the proof is 6 seconds, where as, the verification is 5 milliseconds. It should be noted that the distributor AV can generate multiple the proofs offline before starting the communication session with the responder AVs. Hence, the total computation time needed to run ABE and zk-SNARK verification of our scheme is low, which is suitable for our application because the AVs are in motion and their communication time is short.

In our scheme, blockchain is required to ensure the authenticity and integrity of the new update. To reduce the cost needed to execute our scheme on the blockchain, most of the computations to secure the scheme are done outside the blockchain. Using a consortium blockchain will remove any constraint on the amount of data that should be sent and stored on the blockchain. Also, as discussed before, the computation cost to run our scheme is low, and according to [2], the mean throughput for delivering data to and from moving vehicles that use IEEE 802.11 protocol is equal to 760 kbit/s. If we assume that the size of a firmware update equals to 0.5 MByte, then the time required to transfer the update is 5 seconds. Therefore, given this transfer time and the short time needed for the cryptographic computations, our scheme can be executed during the contact time of two moving AVs.

B. Security analysis

In this section, we discuss the possible security threats and how our scheme mitigate to each of them.

- *Firmware integrity.* Since the authentication code of each new firmware update is recorded in the smart contract, our scheme resists any attempt by an adversary to distribute malicious updates.
- *Firmware distribution and access control.* To prevent unauthorized AVs from accessing a new firmware update, our scheme allows each AV manufacturer to control the access by defining the list of authorized AVs. Through ABE access policy, which is registered in the blockchain, a distributor can prescribe the AVs that have the right to receive the update and prevent unauthorized AVs from receiving it. In addition, because the access policy is

embedded in the update authentication code (AC), any responder AVs (receiver) can decide if it is concerned by a particular update or not. If a compromised/malicious distributor AV changes the access policy used to encrypt the challenge message, the responder AV will detect this change during the zk-SNARK verification. Hence, the distributor (attacker) will not be able to get a proof of distribution from the responder.

- *DoS attack resistance.* The proposed scheme resists Denial-of-service (DoS) attacks that aim to disable the system and the rewarding mechanism. This attack is not possible since there is no central unit that distributes the firmware or runs the scheme. For this attack to succeed, the attackers need to control the majority of the validators (manufacturers) of the blockchain network, which is presumably impossible.
- *Update audibility.* In our scheme, the AVs that have distributed or received an update are recorded in the blockchain. This gives the manufacturer an accurate insight about the firmware state of its AVs, i.e., which AVs have received a particular update.

V. RELATED WORK

In this section, we review some of the existing works that addressed the the security of firmware update .

In [13], Nilsson et al. proposed a firmware update protocol for modern intelligent vehicles over the internet. The authors suggested a client-server method using a web portal that delivers the firmware update in fragments. The fragments are protected using hash chain and ensuring. However, the system is vulnerable to a DoS attack as it relies on a central server. Also, the central solution does not ensure the availability of the updates.

In sensor networks, [14] have been proposed to improve the reliability of delivering new updates/security patches by ensuring their integrity. However, these schemes depend on a single entity to manage the distribution of firmware updates and not scale for large networks.

In [15], the authors proposed a decentralized solution based on a permission-less blockchain to ensure the integrity of updates by having multiple verification nodes instead of depending on a private centralized vendor network. For the distribution of updates, a peer-to-peer file sharing network such as BitTorrent is proposed to ensure integrity and versions tractability of updates. However, the scheme does not provide any incentive for devices to participate and distribute firmware updates to others.

In [16], the authors proposed software update framework for Internet of Things (IoT) devices. The framework allows other parties to deliver the updates in return for digital currency paid by the vendor. However, the scheme incurs high financial cost since it depends on Ethereum blockchain [8] which apply fees for each transaction sent to the network.

VI. CONCLUSION

In this paper, a firmware update scheme based on blockchain and smart contract is introduced for autonomous

vehicles. A smart contract is used to ensure the authenticity and integrity of firmware updates, and more importantly to manage the reputation values of AVs that transfer the new updates to other AVs. We also use ABE to allow AV manufacturer to target a specific set of AVs that have certain features defined by the manufacturer to download the firmware. A zero-knowledge proof protocol is used to enable the AVs to exchange an update for proof of distribution in a trust-less way. Finally, the smart contract rewards the distributors by increasing their reputation in the blockchain. Our evaluation analysis indicates that the cryptography primitives used to secure the firmware update exchange is suitable to the AVs network. For the future work, we will implement a prototype of our proposed scheme.

VII. ACKNOWLEDGEMENT

This work was supported by the US National Science Foundation under Grants 1618549 and 1560434.

REFERENCES

- [1] T. Lassa. The beginning of the end of driving. [Online]. Available: <http://www.motortrend.com/news/the-beginning-of-theend-of-driving/>
- [2] J. Eriksson, H. Balakrishnan, and S. Madden, "Cabernet: vehicular content delivery using wifi," in *Proceedings of the 14th ACM international conference on Mobile computing and networking*. ACM, 2008, pp. 199–210.
- [3] A. Greenberg. [Online]. Available: <https://www.wired.com/2015/07/jeep-hack-chrysler-recalls-1-4m-vehicles-bug-fix/>
- [4] G. Lin, D. Fu, J. Zhu, and G. Dasmalchi, "Cloud computing: It as a service," *IT professional*, no. 2, pp. 10–13, 2009.
- [5] M. Baza, M. Nabil, M. Ismail, M. Mahmoud, E. Serpedin, and M. Rahman, "Blockchain-based privacy-preserving charging coordination mechanism for energy storage units," *arXiv preprint arXiv:1811.02001*, 2018.
- [6] M. Baza, M. Fouda, M. Nabil, A. S. Tag, H. Mansour, and M. Mahmoud, "Blockchain-based distributed key management approach tailored for smart grid," in *Combating Security Challenges in the Age of Big Data*. Springer, 2019, accepted for publication.
- [7] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.
- [8] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum project yellow paper*, vol. 151, pp. 1–32, 2014.
- [9] Y. Rouselakis and B. Waters, "Efficient statically-secure large-universe multi-authority attribute-based encryption," in *Proceedings of the International Conference on Financial Cryptography and Data Security*. Springer, 2015, pp. 315–332.
- [10] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza, "Succinct non-interactive zero knowledge for a von neumann architecture," in *USENIX Security Symposium*, 2014, pp. 781–796.
- [11] J. A. Akinyele, C. Garman, I. Miers, M. W. Pagano, M. Rushanan, M. Green, and A. D. Rubin, "Charm: a framework for rapidly prototyping cryptosystems," *Journal of Cryptographic Engineering*, vol. 3, no. 2, pp. 111–128, 2013.
- [12] Zokrates toolbox. [Online]. Available: <https://github.com/Zokrates/ZoKrates>
- [13] D. K. Nilsson and U. E. Larson, "Secure firmware updates over the air in intelligent vehicles," in *Communications Workshops, 2008. ICC Workshops '08. IEEE International Conference on*. IEEE, 2008, pp. 380–384.
- [14] D. Kim, H. Nam, and D. Kim, "Adaptive code dissemination based on link quality in wireless sensor networks," *IEEE Internet of Things Journal*, vol. 4, no. 3, pp. 685–695, 2017.
- [15] B. Lee and J.-H. Lee, "Blockchain-based secure firmware update for embedded devices in an internet of things environment," *The Journal of Supercomputing*, vol. 73, no. 3, pp. 1152–1167, 2017.
- [16] O. Leiba, Y. Yitzchak, R. Bitton, A. Nadler, and A. Shabtai, "Incentivized delivery network of iot software updates based on trustless proof-of-distribution," *arXiv preprint arXiv:1805.04282*, 2018.